

PERTEMUAN 5 - STACK (LIFO - LEFT IN FIRST OUT)

LAPORAN PRAKTIKUM STRUKTUR DATA PERTEMUAN 5: STACK (LIFO – LAST IN FIRST OUT)

Disusun Oleh:

- Nama: Nadine Valia Azzahra
- NIM: 2C2230007
- PRODI: Sains Data

TUJUAN PRAKTIKUM

1. Memahami konsep dasar struktur data Stack dengan prinsip LIFO (Last In First Out).
2. Mengimplementasikan Stack menggunakan struktur data list pada Python.
3. Memahami operasi dasar Stack seperti push, pop, peek, size, dan pengecekan kondisi kosong.
4. Menganalisis penerapan Stack dalam berbagai kasus sederhana dan menengah.
5. Mengembangkan kemampuan pemrograman dalam mengelola data menggunakan konsep Stack.

DASAR TEORI

Stack merupakan salah satu struktur data linear yang bekerja berdasarkan prinsip Last In First Out (LIFO), yaitu elemen yang terakhir masuk akan menjadi elemen pertama yang keluar. Operasi utama pada Stack meliputi push untuk menambahkan data, pop untuk menghapus data teratas, dan peek untuk melihat elemen teratas tanpa menghapusnya. Struktur data ini banyak digunakan dalam berbagai aplikasi komputer seperti manajemen memori, undo-redo pada editor, evaluasi ekspresi matematika, serta pemanggilan fungsi dalam program. Pada Python, Stack dapat diimplementasikan secara sederhana menggunakan tipe data list dengan memanfaatkan metode `append()` dan `pop()`.

LEVEL MUDAH

```
# TUGAS 1 - Stack Sederhana Menggunakan List
```

```
stack = []
stack.append(10)
stack.append(20)
stack.append(30)
stack.append(40)
stack.append(50)
print(f" Isi Stack : {stack}")
print(f" Jumlah    : {len(stack)} elemen")
```

```
Isi Stack : [10, 20, 30, 40, 50]
Jumlah    : 5 elemen
```

Analisis Hasil:

Program berhasil membuat sebuah Stack menggunakan list Python dan menambahkan lima elemen menggunakan operasi `append()`. Hasil output menunjukkan bahwa seluruh data tersimpan secara berurutan dari bawah ke atas yaitu [10, 20, 30, 40, 50]. Program juga menghitung jumlah elemen menggunakan fungsi `len()` sehingga diperoleh total lima elemen dalam Stack.

```
# TUGAS 2 - Operasi Push dan Pop
```

```
stack_t2 = []
```

```
# Push
for nilai in [100, 200, 300]:
    stack_t2.append(nilai)
    print(f" Push({nilai}) => Stack: {stack_t2}")

# Pop
keluar = stack_t2.pop()
print(f" Pop()      => Keluar: {keluar} | Stack: {stack_t2}")
keluar = stack_t2.pop()
print(f" Pop()      => Keluar: {keluar} | Stack: {stack_t2}")
```

```
Push(100) => Stack: [100]
Push(200) => Stack: [100, 200]
Push(300) => Stack: [100, 200, 300]
Pop()      => Keluar: 300 | Stack: [100, 200]
Pop()      => Keluar: 200 | Stack: [100]
```

▼ Analisis Hasil:

Program melakukan simulasi operasi dasar Stack yaitu push dan pop. Saat nilai 100, 200, dan 300 dimasukkan, data ditempatkan pada bagian atas Stack secara berurutan. Ketika operasi pop() dijalankan, elemen 300 keluar terlebih dahulu kemudian diikuti oleh elemen 200. Hasil ini membuktikan bahwa Stack bekerja sesuai prinsip LIFO, di mana elemen terakhir yang masuk akan menjadi elemen pertama yang keluar.

```
# TUGAS 3 - Tampilkan Elemen Teratas (Peek)

stack = [5, 15, 25, 35]
print(f" Stack          : {stack}")
if stack:
    print(f" Elemen teratas : {stack[-1]}")
else:
    print(" Stack kosong!")
```

```
Stack          : [5, 15, 25, 35]
Elemen teratas : 35
```

Analisis Hasil:

Program menampilkan isi Stack dan mengambil elemen paling atas menggunakan indeks [-1]. Output menunjukkan bahwa elemen teratas dari Stack [5, 15, 25, 35] adalah 35. Operasi ini tidak menghapus data dari Stack sehingga hanya berfungsi untuk melihat elemen teratas.

▼ LEVEL MENENGAH

```
# TUGAS 4 - Menu Stack Interaktif

class StackInteraktif:
    def __init__(self):
        self._data = []

    def push(self, nilai):
        self._data.append(nilai)
        print(f" [PUSH] {nilai} masuk => Stack: {self._data}")

    def pop(self):
        if self.is_empty():
            print(" [POP] Stack kosong, tidak ada data!")
            return None
        keluar = self._data.pop()
        print(f" [POP] {keluar} keluar => Stack: {self._data}")
        return keluar

    def peek(self):
        if self.is_empty():
            print(" [PEEK] Stack kosong!")
            return None
        print(f" [PEEK] Elemen teratas: {self._data[-1]}")
        return self._data[-1]
```

```

def is_empty(self):
    return len(self._data) == 0

def size(self):
    print(f" [SIZE] Jumlah elemen: {len(self._data)}")
    return len(self._data)

def tampilkan(self):
    if self.is_empty():
        print(" [INFO] Stack kosong.")
    else:
        print(f" [INFO] Stack (bawah->atas): {self._data}")

si = StackInteraktif()
si.push("Apel")
si.push("Mangga")
si.push("Jeruk")
si.push("Durian")
si.tampilkan()
si.peek()
si.size()
si.pop()
si.pop()
si.tampilkan()

[PUSH] Apel masuk => Stack: ['Apel']
[PUSH] Mangga masuk => Stack: ['Apel', 'Mangga']
[PUSH] Jeruk masuk => Stack: ['Apel', 'Mangga', 'Jeruk']
[PUSH] Durian masuk => Stack: ['Apel', 'Mangga', 'Jeruk', 'Durian']
[INFO] Stack (bawah->atas): ['Apel', 'Mangga', 'Jeruk', 'Durian']
[PEEK] Elemen teratas: Durian
[SIZE] Jumlah elemen: 4
[POP] Durian keluar => Stack: ['Apel', 'Mangga', 'Jeruk']
[POP] Jeruk keluar => Stack: ['Apel', 'Mangga']
[INFO] Stack (bawah->atas): ['Apel', 'Mangga']

```

▼ Analisis Hasil:

Program mengimplementasikan Stack dalam bentuk class sehingga operasinya menjadi lebih terstruktur. Beberapa data berupa nama buah berhasil ditambahkan menggunakan fungsi push(). Setelah itu program menampilkan isi Stack, mengecek elemen teratas melalui fungsi peek(), menghitung jumlah elemen menggunakan size(), serta menghapus dua elemen teratas menggunakan pop(). Hasil akhir menunjukkan bahwa elemen yang keluar adalah "Durian" dan "Jeruk" sesuai urutan LIFO.

```

# TUGAS 5 - Hitung Jumlah Elemen Stack

stack = []
data_masuk = ["Senin", "Selasa", "Rabu", "Kamis", "Jumat"]
for d in data_masuk:
    stack.append(d)

print(f" Stack          : {stack}")
print(f" Jumlah elemen : {len(stack)}")
print(f" Stack kosong? : {'Ya' if not stack else 'Tidak'}")

Stack          : ['Senin', 'Selasa', 'Rabu', 'Kamis', 'Jumat']
Jumlah elemen : 5
Stack kosong? : Tidak

```

▼ Analisis Hasil:

Program memasukkan lima nama hari ke dalam Stack dan kemudian menampilkan seluruh isi Stack beserta jumlah elemennya. Output menunjukkan bahwa terdapat lima elemen yang tersimpan dan kondisi Stack tidak kosong. Penggunaan fungsi len() memudahkan proses perhitungan jumlah data yang tersimpan dalam Stack secara otomatis.

```

# TUGAS 6 - Simulasi Tumpukan Piring

class TumpukanPiring:
    def __init__(self, kapasitas):
        self._tumpukan = []
        self.kapasitas = kapasitas

```

```

def letakkan(self, piring):
    if len(self._tumpukan) >= self._kapasitas:
        print(f" Tumpukan penuh! Tidak bisa letakkan '{piring}'.")
        return
    self._tumpukan.append(piring)
    print(f" Letakkan: {piring} => {self._tumpukan}")

def ambil(self):
    if not self._tumpukan:
        print(" Tumpukan kosong!")
        return None
    piring = self._tumpukan.pop()
    print(f" Ambil : {piring} => Sisa: {self._tumpukan}")
    return piring

def lihat_teratas(self):
    if not self._tumpukan:
        print(" Tumpukan kosong!")
        return
    print(f" Piring teratas: {self._tumpukan[-1]}")

def status(self):
    print(f" Status: {len(self._tumpukan)}/{self._kapasitas} piring")

rak_piring = TumpukanPiring(kapasitas=5)
for i in range(1, 7):
    rak_piring.letakkan(f"Piring-{i}")
rak_piring.status()
rak_piring.lihat_teratas()
rak_piring.ambil()
rak_piring.ambil()
rak_piring.ambil()
rak_piring.status()

Letakkan: Piring-1 => ['Piring-1']
Letakkan: Piring-2 => ['Piring-1', 'Piring-2']
Letakkan: Piring-3 => ['Piring-1', 'Piring-2', 'Piring-3']
Letakkan: Piring-4 => ['Piring-1', 'Piring-2', 'Piring-3', 'Piring-4']
Letakkan: Piring-5 => ['Piring-1', 'Piring-2', 'Piring-3', 'Piring-4', 'Piring-5']
Tumpukan penuh! Tidak bisa letakkan 'Piring-6'.
Status: 5/5 piring
Piring teratas: Piring-5
Ambil : Piring-5 => Sisa: ['Piring-1', 'Piring-2', 'Piring-3', 'Piring-4']
Ambil : Piring-4 => Sisa: ['Piring-1', 'Piring-2', 'Piring-3']
Ambil : Piring-3 => Sisa: ['Piring-1', 'Piring-2']
Status: 2/5 piring

```

Analisis Hasil:

Program mensimulasikan penggunaan Stack dalam kehidupan sehari-hari melalui kasus tumpukan piring. Kapasitas Stack ditetapkan sebanyak lima piring. Ketika piring keenam ditambahkan, program menolak proses tersebut karena kapasitas telah penuh. Selanjutnya dilakukan tiga kali pengambilan piring dan hasil menunjukkan bahwa piring yang terakhir diletakkan akan diambil terlebih dahulu.

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Stack merupakan struktur data linear yang menerapkan prinsip Last In First Out (LIFO). Operasi dasar seperti push, pop, peek, size, dan pengecekan kondisi kosong dapat diimplementasikan dengan mudah menggunakan list maupun class pada Python. Hasil pengujian menunjukkan bahwa elemen yang terakhir dimasukkan selalu menjadi elemen pertama yang dikeluarkan. Selain itu, Stack dapat diterapkan dalam berbagai kasus nyata seperti pengelolaan data, sistem undo-redo, dan simulasi tumpukan piring. Dengan demikian, pemahaman terhadap Stack sangat penting sebagai dasar dalam mempelajari struktur data yang lebih kompleks.

DISKUSI

1. Apa yang dimaksud dengan Stack?

Stack (tumpukan) adalah struktur data linear yang bekerja berdasarkan prinsip LIFO (Last In First Out). Artinya, elemen yang paling terakhir dimasukkan ke dalam tumpukan akan menjadi elemen yang pertama kali dikeluarkan. Analoginya seperti tumpukan piring di atas meja, di mana kita hanya bisa menaruh atau mengambil piring dari bagian paling atas.

2. Mengapa Stack menggunakan prinsip LIFO?

Stack menggunakan prinsip LIFO karena akses datanya sengaja dibatasi hanya pada satu ujung saja, yaitu bagian teratas (Top). Desain ini sangat berguna untuk memproses data yang urutan pengerjaannya harus dibalik (urutan kronologis terbalik), di mana aktivitas atau data paling baru/paling atas harus diselesaikan terlebih dahulu sebelum mengakses data di bawahnya.

3. Apa perbedaan Push dan Pop?

Push: Operasi untuk menambahkan atau memasukkan elemen baru ke posisi paling atas (Top) dari stack. Operasi ini meningkatkan ukuran tumpukan sebesar 1.

Pop: Operasi untuk mengambil, menghapus, atau mengeluarkan elemen yang berada di posisi paling atas (Top) dari stack. Operasi ini mengurangi ukuran tumpukan sebesar 1.

4. Mengapa operasi Stack memiliki kompleksitas $O(1)$?

Operasi dasar pada stack seperti Push, Pop, dan Peek memiliki kompleksitas waktu konstan atau $O(1)$ karena operasi tersebut langsung dilakukan pada ujung atas (Top) tumpukan tanpa memerlukan penelusuran (traversing) atau pergeseran elemen-elemen lain di dalam memori. Indeks elemen teratas selalu dapat diketahui secara langsung.

5. Apa fungsi Call Stack?

Call Stack adalah mekanisme runtime yang digunakan oleh sistem komputer atau interpreter pemrograman untuk melacak pemanggilan fungsi (subroutine) dalam suatu program. Fungsi utamanya adalah mencatat titik kembali (return address) program, sehingga ketika sebuah fungsi selesai dieksekusi, sistem tahu ke mana harus mengembalikan alur kendali eksekusi program.

6. Apa hubungan Stack dengan Rekursi?

Rekursi sangat bergantung pada struktur data stack. Setiap kali sebuah fungsi memanggil dirinya sendiri, sistem operasi akan membungkus parameter, variabel lokal, dan alamat kembali fungsi tersebut ke dalam sebuah stack frame dan menaruhnya ke dalam call stack sistem. Ketika kondisi batas (basis rekursi) tercapai, fungsi-fungsi tersebut akan diselesaikan dan dikeluarkan (pop) dari stack satu per satu dalam urutan terbalik.

7. Apa yang menyebabkan Stack Overflow?

Stack Overflow terjadi ketika memori stack yang dialokasikan untuk program telah penuh, tetapi program masih mencoba memasukkan data baru ke dalamnya. Penyebab paling umum adalah terjadinya fungsi rekursif tanpa henti (infinite recursion) akibat tidak adanya kondisi batas (base case) yang valid, atau pemanggilan fungsi yang terlalu dalam sehingga menghabiskan ruang memori call stack.

8. Mengapa fitur Undo menggunakan Stack?

Fitur Undo dirancang untuk membatalkan aksi terakhir yang dilakukan oleh pengguna. Karena aksi yang paling terakhir dilakukan adalah yang harus pertama kali dibatalkan, maka mekanisme LIFO pada stack sangat ideal untuk kasus ini. Setiap aksi pengguna akan di-push ke dalam stack, dan ketika pengguna melakukan Undo, aksi teratas akan di-pop untuk membatalkan perubahan tersebut.

9. Apa kelebihan dan kekurangan Stack?

Kelebihan:

- Sederhana dan sangat mudah diimplementasikan.
- Sangat efisien dalam penggunaan memori dan waktu eksekusi (kompleksitas operasi dasar selalu $O(1)$).
- Sangat cocok untuk manajemen memori fungsi dan fitur pelacakan mundur (backtracking).

Kekurangan:

- Ukurannya terbatas (berisiko terjadi stack overflow jika tumpukan terlalu besar).
- Tidak mendukung akses acak (random access). Tidak bisa membaca atau memodifikasi data di tengah tumpukan tanpa mengeluarkan elemen-elemen di atasnya terlebih dahulu.

10. Sebutkan minimal 5 penerapan Stack dalam kehidupan nyata dan sistem komputer.

1. Fitur Undo dan Redo pada aplikasi penyunting teks (seperti MS Word, VS Code, atau Notepad).

2. Tombol Back pada Web Browser, yang menyimpan riwayat URL halaman web yang kita kunjungi untuk kembali ke halaman sebelumnya.
3. Call Stack pada Runtime Engine, yang digunakan untuk mengatur pemanggilan fungsi di memori komputer.
4. Evaluasi Ekspresi Matematika, seperti proses konversi notasi matematika Infix menjadi Postfix atau Prefix pada kalkulator.
5. Algoritma Depth-First Search (DFS), yaitu algoritma pencarian atau penelusuran struktur data graf/pohon.
6. Tumpukan Fisik di Dunia Nyata, seperti tumpukan piring di dapur, tumpukan buku di perpustakaan, atau tumpukan koin.